

RECAP – Turingmaschinen

- Wir unterscheiden zwischen **total rekursiv (entscheidbar)**, **rekursiv (entscheidbar)** und **rekursiv aufzählbar (semi-entscheidbar)**
- Eine Konfiguration $\alpha q \beta$ beschreibt eindeutig den aktuellen Systemzustand einer TM
- In einem Rechenschritt erreichen wir eine direkte Nachfolgekonfiguration, in endlich vielen eine Nachfolgekonfiguration.
- Mehrspurmaschinen und Mehrbandmaschinen erleichtern das Programmieren einer TM. Sie sind aber nicht mächtiger, als eine „klassische“ TM
- D.h. sie lassen sich mit einem gewissen Aufwand simulieren.

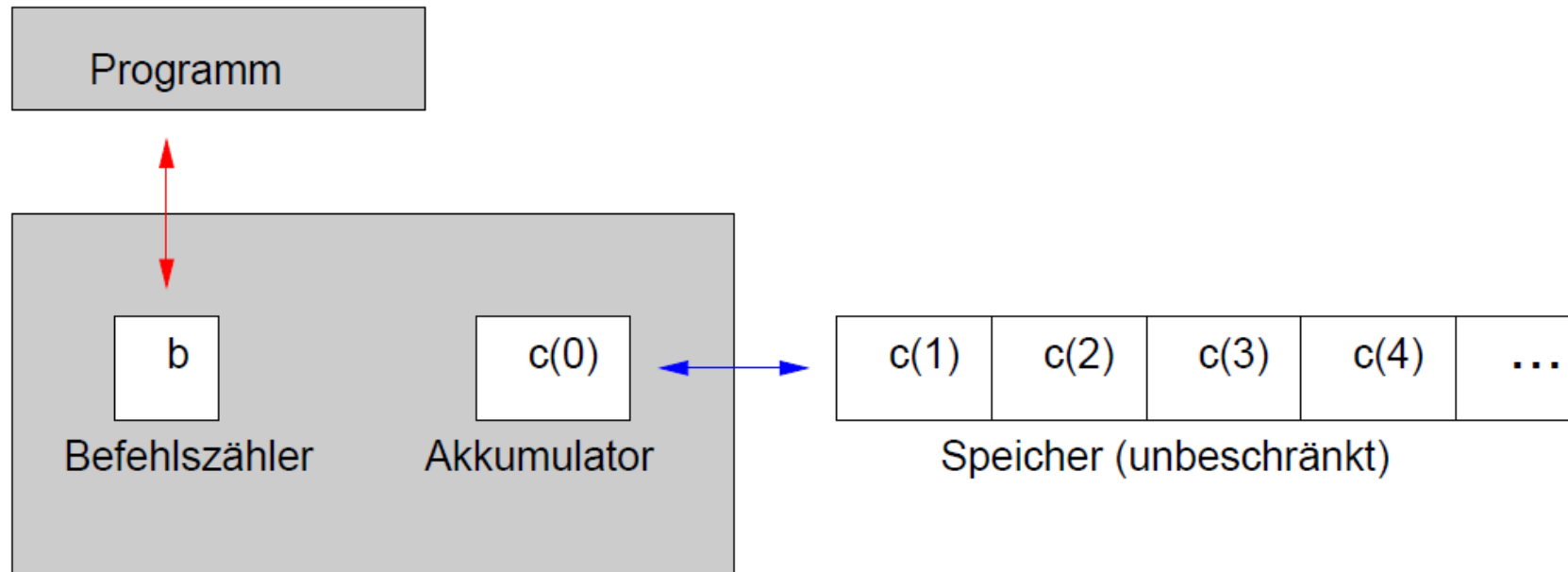
Simulation RAM $\Leftrightarrow$ TM

Simulation von RAMs durch TMs

Satz: Jede im logarithmischen Kostenmaß $t(n)$ -zeitbeschränkte RAM kann für ein Polynom q durch eine $O(q(n + t(n)))$ -zeitbeschränkte TM simuliert werden.

Beweis: Im Beweis können wir für die Simulation eine 2-Band-TM statt einer (1-Band) TM verwenden. (Warum?)

Registermaschine (genauer: Random Access Machine; RAM)



Befehlsatz:

LOAD, STORE, ADD, SUB, DIV

INDLOAD, INDSTORE, INDADD, INDSUB, INDDIV

CLOAD, CADD, CSUB, CDIV

GOTO, IF c(0)?x THEN GOTO j (wobei ? aus {=, <, <=, >, >=}), END

Simulation von RAMs durch TMs

Beweis (1):

- Wir zeigen, dass die Laufzeit der Simulation der RAM mit Laufzeitschranke $t(n)$ durch eine 2-Band TM nach oben beschränkt ist durch $t'(n) = \alpha(n + t(n))^\beta$.
- Nach letztem Satz können wir dies mit einer 1-Band TM mit Laufzeitschranke $t''(n) = \gamma(t'(n))^2$ simulieren.
- Für die Simulation einer RAM durch eine 1-Band TM ergibt sich somit eine Laufzeitschranke $t''(n) = \gamma\alpha^2(n + t(n))^{2\beta}$. Dies ist polynomiell in $t(n)$.

Simulation von RAMs durch TMs

Beweis (intermezzo):

- Wichtige Beobachtung: Die Klasse der Polynome ist gegen Komposition abgeschlossen.
- Deshalb können wir eine konstante Anzahl von Simulationen, deren Zeitverlust jeweils polynomiell nach oben beschränkt ist, ineinander schachteln und erhalten dadurch wiederum eine Simulation mit polynomiell beschränktem Zeitverlust.

Simulation von RAMs durch TMs

Beweis (2):

- Das Programm P der RAM bestehe aus p Programmzeilen. Für jede Programmzeile schreiben wir ein TM-Unterprogramm. Sei M_i das Unterprogramm für Programmzeile i , $0 \leq i \leq p$.
- Außerdem spezifizieren wir ein Unterprogramm M' für die Initialisierung der TM und M_{p+1} für die Aufbereitung der Ausgabe des Ergebnisses.
- Die Registermaschinenkonfiguration wird durch den Befehlszähler b und den Inhalt der Register beschrieben.
 - Den Befehlszähler $b \in \{1, \dots, p\}$ kann die TM im Zustand abspeichern, da p konstant ist (d.h. unabhängig von der Eingabe).
 - Der Inhalt der benutzen Register ist allerdings nicht durch eine Konstante beschränkt.

Simulation von RAMs durch TMs

Beweis (3):

- Speicherung der Register auf einem Band der TM:
 - Seien $0, i_1, \dots, i_m$ die zu einem Zeitpunkt benutzten Register, also die Indizes des Akkumulators und derjenigen anderen Speicherstellen, die Teile der Eingabe enthalten oder in einem der vorhergehenden Rechenschritte angesprochen wurden.
 - Wir verwenden eine 2-Band-TM und speichern diese Indizes und den Inhalt der entsprechenden Register in der folgenden Form auf Band 2 ab:

$$\#\#0\#\text{bin}(c(0))\#\#\text{bin}(i_1)\#\text{bin}(c(i_1))\#\# \dots \#\#\text{bin}(i_m)\#\text{bin}(c(i_m))\#\#\#$$
- Die Länge des Speicherinhalts auf Band 2 ist durch $O(n + t(n))$ beschränkt, weil die RAM für jedes neue Bit, das sie erzeugt, mindestens eine Zeiteinheit benötigt.

Simulation von RAMs durch TMs

Beweis (4):

- Rechenschritt für Rechenschritt simuliert die TM nun die Konfigurationsveränderungen der RAM. Dazu ruft es das durch den Programmzähler b beschriebene Unterprogramm M_b auf.
- Das Unterprogramm M_b
 - kopiert den Inhalt der in Programmzeile b angesprochenen Register auf Band 1,
 - führt die notwendigen Operationen auf diesen Registerinhalten durch,
 - kopiert dann das Ergebnis in das in Zeile b angegebene Register auf Band 2 zurück, und
 - aktualisiert zuletzt den Programmzähler b .

Simulation von RAMs durch TMs

Beweis (5):

- Laufzeitanalyse:
 - Die Initialisierung erfordert Zeit $O(n)$.
 - Alle anderen Unterprogramme haben eine Laufzeit, die polynomiell in der Länge der Bandinschrift auf Band 2 beschränkt ist, also eine polynomielle Laufzeit in $n + t(n)$.
 - Die Gesamtlaufzeit der Simulation ist somit auch polynomiell in $n + t(n)$ beschränkt.

QED

Simulation TM durch RAM

- **Satz:** Jede $t(n)$ -zeitbeschränkte TM kann durch eine RAM simuliert werden, die uniform $O(t(n) + n)$ und logarithmisch $O((t(n) + n) \log(t(n) + n))$ zeitbeschränkt ist.
- **Folgerung:** 1-Band TM, Mehrband-TM und RAM können sich gegenseitig mit polynomiellen Zeitverlust simulieren. Wenn es uns also „nur“ um Fragen der Berechenbarkeit von Problemen oder um ihre Lösbarkeit in polynomieller Zeit geht, können wir wahlweise auf die TM, die Mehrband-TM oder die RAM zurückgreifen.
- Für positive Ergebnisse (wie etwa obere Laufzeitschranken) ist es häufig sinnvoll Algorithmen für die RAM zu spezifizieren.
- Für den Nachweis negativer Ergebnisse werden wir meistens auf die TM zurückgreifen, da sie das strukturell einfachste unter diesen Rechenmodellen ist.

Simulation TM durch RAM

Beweis (1):

- Wie schon bei den anderen Simulationen müssen wir die Konfiguration der simulierten Maschine (in diesem Fall der TM) im Speicher der simulierenden Maschine (der RAM) abbilden.
 - OBdA nehmen wir an, es handelt sich um eine TM mit einseitig beschränktem Band, deren Zellen mit 0, 1, 2, ... durchnummeriert sind.
 - Die Zustände und Zeichen werden ebenfalls durchnummeriert und mit ihren Nummern identifiziert, so dass sie in den Registern abgespeichert werden können.
 - Register 2 speichert den aktuellen Zustand.
 - Register 1 speichert den Index der Kopfposition.
 - Die Register 3, 4, ... speichern die Inhalte der Bandpositionen 0, 1, 2, ... soweit diese Positionen zur Eingabe der TM gehören oder zuvor vom Kopf der TM besucht worden sind.

Simulation TM durch RAM

Beweis (2):

- Die TM wird nun Schritt für Schritt durch die RAM simuliert.
- Zunächst wird überprüft, ob die TM im aktuellen Schritt terminiert. Falls ja terminiert auch die RAM mit der entsprechenden Ausgabe.
- Ansonsten muss der Zustandübergang durchgeführt werden:
 - Auf einer ersten Stufe von $|Q|$ vielen if-Abfragen wird dazu der aktuelle Zustand selektiert.
 - Für jeden möglichen Zustand, gibt es dann eine zweite Stufe von $|\Gamma|$ vielen if-Abfragen, die das gelesene Zeichen selektieren.
 - Je nach Ausgang der if-Abfragen wird dann der Zustand in Register 2 geändert, die Bandinschrift im Register mit der Adresse $c(1)$ überschrieben, und die Bandposition in Register 1 aktualisiert.

Simulation TM durch RAM

Beweis (3):

- Laufzeitanalyse im uniformen Kostenmodell:
 - Die Initialisierung kann in Zeit $O(n)$ durchgeführt werden.
 - Die Simulation jedes einzelnen TM-Schrittes hat konstante Laufzeit.
 - Insgesamt ist die Simulationszeit somit $O(n + t(n))$.

Simulation TM durch RAM

Beweis (4):

- Laufzeitanalyse im logarithmischen Kostenmodell:
 - Die in den Registern gespeicherten Zahlen repräsentieren Zustände, Zeichen und Bandpositionen.
 - Zustände und Zeichen haben eine konstante Kodierungslänge.
 - Die Bandpositionen, die während der Simulation angesprochen werden, sind durch $\max(n, t(n)) \leq n + t(n)$ beschränkt. Die Kodierungslänge dieser Positionen ist also $O(\log(t(n) + n))$.
 - Damit kann die Simulation jedes einzelnen TM-Schrittes in Zeit $O(\log(t(n) + n))$ durchgeführt werden.
 - Insgesamt ergibt sich somit eine Simulationszeit von $O(\log(t(n) + n) \cdot (t(n) + n))$

QED

Universelle Turingmaschinen

Universelle TMs

- Bisher haben wir für jedes Problem eine eigene TM entworfen, einen special purpose Rechner.
- Real existierende Maschinen sind jedoch programmierbare general purpose Rechner.
- Wir konstruieren jetzt eine programmierbare Variante der TM, die sogenannte universelle TM.

Universelle TMs

- Das Programm der **universellen Turingmaschine** U ist die Kodierung einer beliebigen TM M .
- Diese Kodierung heißt die **Gödelnummer** von M und wird mit $\langle M \rangle$ bezeichnet.
- Als Eingabe erhält U einen String der Form $\langle M \rangle w$ bestehend aus der Gödelnummer $\langle M \rangle$ und einem beliebigen Wort w .
- Die universelle TM simuliert das Verhalten der TM M auf der Eingabe w .
- Bei inkorrektter Eingabe (d.h. die Eingabe beginnt nicht mit einer Gödelnummer) gibt U eine Fehlermeldung aus.

Universelle TMs

- Mit **Gödelnummern** bezeichnet man die eindeutige prefixfreie Kodierung von TM über einem festen Alphabet.
- OBdA gehen wir von binären Kodierungen über dem Alphabet $\{0,1\}$ aus.
- **Prefixfrei** bedeutet, dass keine Gödelnummer Prefix (Anfangsteilwort) einer anderen Gödelnummer sein darf.
 - Wichtig um $\langle M \rangle w$ in M und w zerlegen zu können.
- Prefixfreiheit können wir beispielsweise erreichen indem alle Gödelnummern auf 111 enden und ansonsten der Teilstring 111 nicht in der Kodierung vorkommt oder alternativ alle Gödelnummern mit 111 beginnen und auf 111 enden und ansonsten der Teilstring 111 nicht in der Kodierung vorkommt.

Universelle TMs

- Wir zeigen, wie eine geeignete Kodierung von TM aussehen könnte.
- OBdA beschränken wir uns auf TM der folgenden Form:
 - Sei $Q = \{q_1, \dots, q_t\}$
 - Der Anfangszustand sei q_1 und es gibt oBdA nur einen akzeptierenden Endzustand q_2 .
 - OBdA sei $\Gamma = \{0, 1, B\}$. Wir nummerieren das Alphabet durch, indem wir $X_1 = 0, X_2 = 1, X_3 = B$ setzen.
 - Auch die möglichen Kopfbewegungen nummerieren wir indem wir $D_1 = L, D_2 = R, D_3 = N$ setzen.
- Zur Beschreibung von TM dieser Form müssen wir nur die Übergangsfunktion als Binärstring kodieren.

Universelle TMs

- Kodierung der Übergangsfunktion:
 - Der Übergang $\delta(q_i, X_j) = (q_k, X_l, D_m)$ wird kodiert durch den Binärstring $0^i 10^j 10^k 10^l 10^m$
 - Die Kodierung des j-ten Übergangs bezeichnen wir mit $\text{code}(j)$.
 - Die Gödelnummer einer TM M mit s vielen Übergängen ist dann $\langle M \rangle = 111\text{code}(1)11\text{code}(2)11 \dots 11\text{code}(s)111$.

Universelle TMs

- Mögliche Implementierung der universellen TM (Teil 1)
 - Als Eingabe erhält die universelle TM U ein Wort der Form $\langle M \rangle w$ für beliebiges $w \in \{0,1\}^*$.
 - Wir implementieren U zunächst in Form einer 3-Band TM:
 - Band 1 von U simuliert das Band der TM M .
 - Band 2 von U enthält die Gödelnummer von M .
 - Auf Band 3 speichert U den jeweils aktuellen Zustand von M .

Universelle TMs

- Mögliche Implementierung der universellen TM (Teil 2)
 - Initialisierung:
 - U überprüft, ob die Eingabe mit einer korrekten Gödelnummer beginnt. Falls nein, Fehlerausgabe.
 - U kopiert die Gödelnummer auf Band 2 und schreibt die Binärkodierung des Anfangszustands auf Band 3.
 - U bereitet Band 1 so vor, dass es nur das Wort w enthält.
 - Der Kopf steht unter dem ersten Zeichen von w .
 - Laufzeit? Die Laufzeit ist $O(1)$, wobei wir die Kodierungslänge von M als Konstante ansehen.

Universelle TMs

- Mögliche Implementierung der universellen TM (Teil 3)
 - Simulierung eines Schritts von M :
 - U sucht zu dem Zeichen an der Kopfposition aus Band 1 und dem Zustand auf Band 3 die Kodierung des entsprechenden Übergangs von M auf Band 2.
 - Wie in der Übergangsfunktion beschrieben aktualisiert U die Inschrift auf Band 1, bewegt U den Kopf auf Band 1, und verändert U den auf Band 3 abgespeicherten Zustand von M .
 - Laufzeit eines Schrittes: $O(1)$
 - Das bedeutet U simuliert M mit konstantem Zeitverlust!

Universelle TMs

- Mögliche Implementierung der universellen TM (Teil 4)
 - Können wir dieses Ergebnis auch mit einer (1-Band) TM erreichen?
 - Natürlich können wir die beschriebene 3-Band TM auf der 1-Band TM mit mehreren Spuren simulieren.
 - Aber bei Verwendung der Simulation handeln wir uns einen quadratischen Zeitverlust ein.
 - Eine universelle 1-Band TM mit konstantem Zeitverlust erhalten wir, wenn wir Gödelnummer und Zustand von M jeweils auf einer Spur abspeichern und diese Inschriften, die ja gemäß unseren Vereinbarungen konstante Länge haben, mit dem Kopf der TM M mitführen, d.h. von Schritt zu Schritt umkopieren, so dass sie immer genau unter dem Kopf stehen.

Wie haben wir Simulationen aufgeschrieben?

Ein simulierendes Rechenmodell M' simuliert ein Modell M .

- 1) Beschreibe M' , insbesondere die Konfiguration von M' vor jedem zu simulierenden Schritt und wie man die Konfiguration von M darin wiederfindet.
- 2) Beschreibe die Initialisierung des Ganzen aus der Eingabe von M .
- 3) Beschreibe im Detail die Simulation eines Rechenschrittes von M durch M'
- 4) [Entfällt bei Entscheidungsproblemen] Beschreibe die Aufbereitung des Outputs

Church-Turing-These

- Die Klasse der Turing-berechenbaren Funktionen stimmt mit der Klasse der intuitiv berechenbaren Funktionen überein.
 - Passt für
 - TMs (und Varianten)
 - RAMs
 - “Normale Computer”
 - Rechnung mit Stift und Papier
 - Quantencomputer
 - λ -Kalkül
 - GOTO-Programme
 - WHILE-Programme
 - Diverse Formen der konstruktiven Mathematik
 - Kann nicht bewiesen werden, aber durchaus widerlegt werden.
 - Relativ stark akzeptiert
 - U.a. da “unsere Physik” (Newton, Maxwell, Relativität, QED, Standardmodell, übliche Stringtheorien) durch Turingmaschinen simuliert werden können. Insbesondere müsste dann “unsere Physik” unvollständig oder falsch sein, damit die These widerlegt ist.

Starke Church-Turing-These

- Die Klasse der Turing-berechenbaren Funktionen stimmt mit der Klasse der intuitiv berechenbaren Funktionen überein und zwei verschiedene Modelle können sich mit polynomielltem Verlust simulieren.
 - Passt für viele Modelle, aber vermutlich nicht für Quantencomputer.
 - Kann nicht bewiesen werden, aber durchaus widerlegt werden.